

IT-Security in the Lifecycle of Products

IT-Sicherheit | 12.02.2026

Gerald Dißbauer | Lecturer

A-SIT Secure Information Technology Center Austria

www.dhbw-stuttgart.de

Agenda

Introduction

IT Security in the Lifecycle of Software-intensive Products

Special Topics on Industrial Secure Lifecycle Practices

Practical Observations from Conducted Projects

Discussions & Summary

Agenda

Introduction

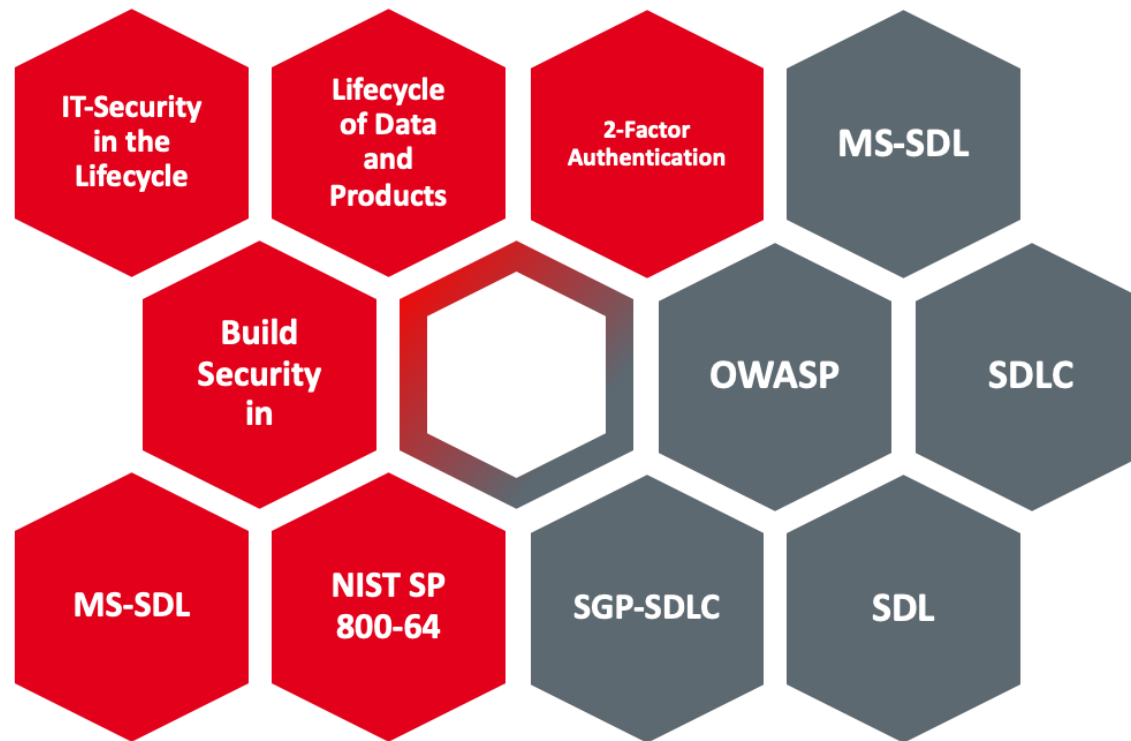
IT Security in the Lifecycle of Software-intensive Products

Special Topics on Industrial Secure Lifecycle Practices

Practical Observations from Conducted Projects

Discussions & Summary

KEYWORDS



RECENT OBSERVATIONS

SECURITY NEEDS TO BE INBUILT TO COUNTER ATTACKS

- 10/2008 | “**Security in the System Development Life Cycle** is essential to implementing and integrating a comprehensive strategy for **managing risk** for all Information Technology assets in an organization”¹
- 10/2009 | “an application needs to be security aware so that it can protect itself from security threats. This implies [...] **security needs to be inbuilt** within the application”²
- 05/2014 | “**Big Data** has complexities that [...] people and companies are **unprepared** to deal with. [...] Those ...] include **security and governance** [...]”⁴

¹ cf. (NIST SP 800-64, 2008, p. 2)

² cf. (Talukder et al., 2009, p. 1)

³ cf. (Paryasto et al., 2014, p. 60)

THE THREAT LANDSCAPE CHANGES

SECURITY IN THE SOFTWARE LIFECYCLE

- Arms race in cyberspace (*e.g. like the cold war*)
- Attacking IoT-devices & threats to autonomous EVs (*e.g. botnets, DDoS*)
- Cyber-terrorism & organized crime (*e.g. monetizing cyber-crime-efforts*)
- Espionage & information disclosure (*e.g. Insider threats, leaks, Tailored-Access-Operations*)
- Supply chain attacks¹ (*e.g. automated software updates as risks, purchase*)
- Increasing investment risk & attacking online financial systems
- Vulnerabilities and patching¹ (*e.g. ICS², libraries, replacing signed code*)

Integrate security control in each SDLC phase

¹ cf. (Cisco, 2018)

² ICS – Industrial Control System

HOW TO BUILD SECURITY IN?

TAKE IT-SECURITY INTO ACCOUNT IN THE LIFECYCLE

- *Data protection by design* and *data protection by default*¹ require security considerations at early phases (*e.g. from the beginning*)
- Design and integrate both security and functionality into the software development lifecycle (SDLC) phases and create secure products
- Understand the risks, the (security) requirements, the goals and the constraints to build (*or acquire*) appropriate computer systems
- Make informed decisions to construct reliable software or processes
- Operate, maintain and dispose software-intensive systems trustworthy

¹ cf. REGULATION (EU) 2016/679 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)

WHICH KIND OF PRODUCTS?¹

SELECTED EXAMPLES OF REAL-WORLD PRODUCTS

- **Automotive Products.** *(e.g., Autonomous EV², Classic Cars, Trucks)*
- **Consumer Goods.** *(e.g., Electric Razors)*
- **Cryptographic Products.** *(e.g., Crypto-Chips, HSM³, HW⁴-Token, SCD⁵)*
- **Data.** *(e.g., Financial Data, Personal Data, Source-Code, Streaming)*
- **Financial Products.** *(e.g., Credit Cards, Insurance, Securities)*
- **Groceries.** *(e.g., Smoothies, Snacks, Soda, Water)*
- **Software-intensive Systems.** *(Information Systems, Embedded Systems – e.g., Medical Information Systems – MIS)*

¹ Annotation: This selection does not restrict relevant products to the examples given.

² EV – Electric Vehicle

³ HSM – Hardware-Security-Module

⁴ HW – Hardware

⁵ SCD – Signature-Creation-Device

SOFTWARE-INTENSIVE PRODUCTS

SOFTWARE AND HARDWARE AS A PRODUCT

- **Product**

“A product is something, that can be offered to a market that might satisfy a want or a (e.g. business) need.”¹

- **Production**

“Production is a process of combining various material inputs and immaterial inputs (e.g. plans, know-how) in order to make something for consumption (i.e. the output). It is the act of creating output, a good or service that has a value and that contributes to the utility of individuals.”^{1,2}

- **Software as a Product (SaaP)**

“SaaP means that the company delivers a copy of the software to the customer, who gets usage rights but not ownership.”³

¹ cf. (Kotler et al., 2006)

² Annotation: It creates an added-value. That means a value-added process is applied.

³ cf. (Popp, 2011, p. 27)

Agenda

Introduction

IT Security in the Lifecycle of Software-intensive Products

Special Topics on Industrial Secure Lifecycle Practices

Practical Observations from Conducted Projects

Discussions & Summary

SECURE SOFTWARE LIFECYCLE

PERFECT SECURITY IS VERY, VERY HARD TO ACHIEVE¹

- Ask the right questions at the right time to understand threats¹
- Build defensible software product quality
- Early investment in security affects the degree of trust of products
- Engineer secure solutions to achieve documented objectives
- Integrate and apply security practices to the development lifecycle
- Software vulnerabilities are preventable

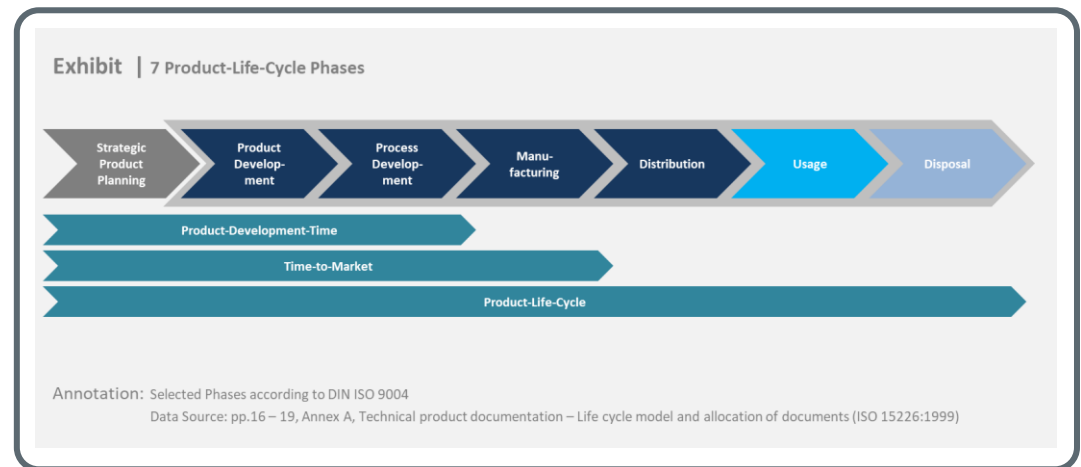
Software products can be secure to a certain degree²

¹ Annotation: Moving targets are hard to hit – i.e., the threats are changing/evolving

² Annotation: Certain degree means that 100% security is hard to achieve both in terms of complexity theory and in practice

PRODUCTS HAVE A LIFECYCLE TAKE SECURITY INTO ACCOUNT

- DIN ISO 9004
- 7 Phases
- Varying duration
- Tailored to companies and products



- **Product-Life-Cycle**
“In most cases, there will be differences from phases of the enterprise-specific [or product-specific] life cycle [...]”¹

¹ cf. (ISO 15226, pp. 16-19)

WHAT ARE SOFTWARE-INTENSIVE SYSTEMS?

SEGMENTATION INTO 2 CATEGORIES

System „combination of interacting elements organized to achieve one or more stated purposes“¹

Software-intensive Systems²

- **Information-Systems.** Information systems (1) capture, (2) store, (3) transform, (4) transmit, and / or (5) process information. The goal is to provide users (*e.g. people or systems*) with desired data/information. It is predominantly based on “Software” (*e.g. system to derive digital signatures*).
- **Embedded-Systems.** Software is – „just“ – an important component of embedded systems (*e.g. ABS³, ESC⁴*). It is integrated into particular hardware and it interacts with e.g. (electro-) mechanical components.

¹ cf. (ISO/IEC/IEEE 15288, p. 9)

² cf. (Pohl, 2008, p. 5)

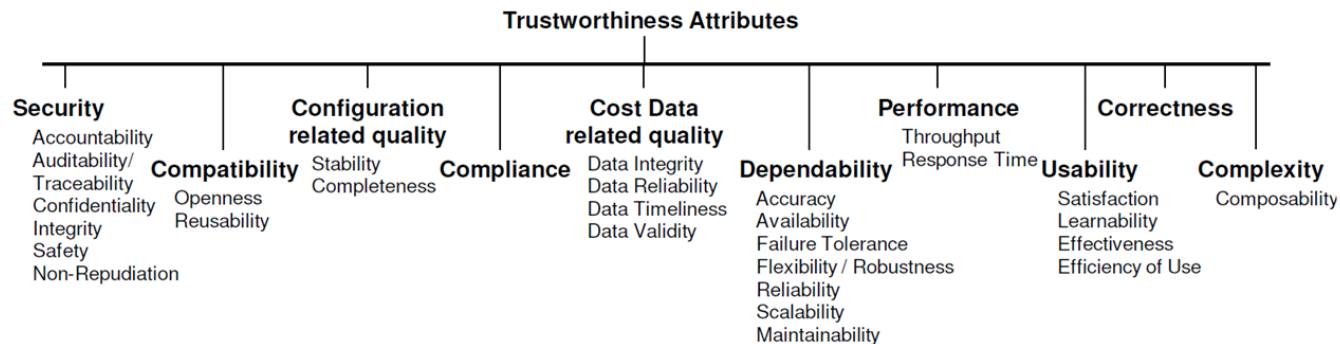
³ ABS – Anti-lock braking system

⁴ ESC – Electronic stability control

TRUTWORTHINESS ATTRIBUTES

TAXONOMY | 10 SELECTED CATEGORIES

- Objective criteria and measures to assess system-trustworthiness (*i.e., no exploitable, maliciously or unintentionally vulnerabilities exist*)²
- Taxonomy¹ of *Trustworthiness Attributes*³



Determine the trustworthiness of software during development

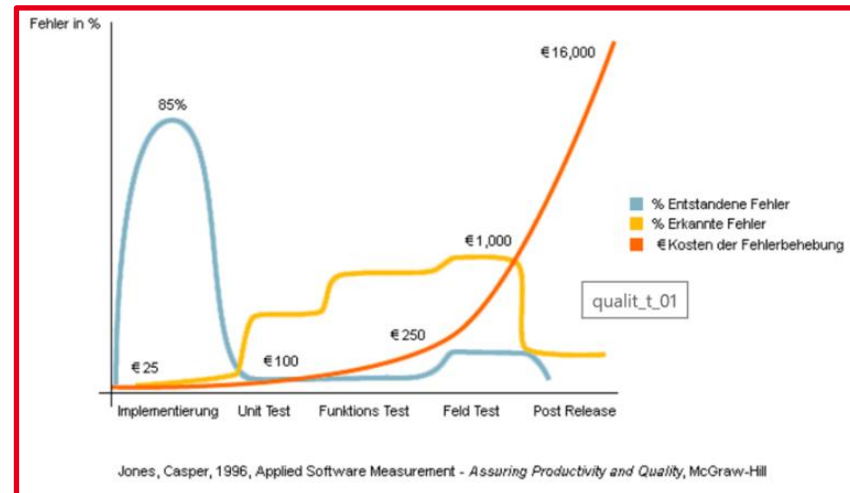
¹ Taxonomy – Taxonomic schemes are methods for classification of objects

² cf. (Department of Homeland Security, 2006, p. ii)

³ cf. (Paulus, Mohammadi and Weyer, 2013, p. 236)

APPLY SECURITY CONCEPTS TO THE SDLC EARLY AND CONTINUOUSLY

- All roles in the SDLC have security responsibility
- Integrate targeted security measures over the SDLC to be effective
- Security features might affect the application/system development
- Vulnerabilities are more expensive to fix the later they are detected in the SDLC
- *“The sooner security measures find their way into the SDLC, the cheaper their implementation or their effect on the construction of secure software becomes.”¹*



¹ cf. (Paulus, 2011, p. 51)

SECURITY DESIGN GUIDELINES BY SALTZER AND SCHROEDER¹

- 8 classic principles in terms of security design
 - **Economy of mechanism.** Keep it short, simple, stupid
 - **Fail-safe defaults.** Avoid single points of failure
 - **Complete mediation.** Check everything and check it permanently
 - **Open design.** Assume you cannot keep the design secret
 - **Separation of privilege.** Separate and use multiple conditions
 - **Least privilege.** Apply privileges absolutely not beyond the need
 - **Least common mechanism.** Avoid shared resources
 - **Psychological acceptability.** How usable and acceptable are security controls

¹ cf. (Saltzer and Schroeder, 1975)

SECURE SOFTWARE DEVELOPMENT LIFECYCLE

GENERAL OBSERVATIONS

- The lifecycle to develop software (*SDLC*^{1,2}) deals with the engineering and applying of both repeatable and predictable processes to deliver software products that meet the requirements³ (*e.g. functionality, needed quality, on-time, budgetary and legal constraints*)
- Apply distinct SDLC-Phases and apply not only security-centered models
- Various SDLC-Approaches exist^{4,5} and are defined by enterprise policies (*e.g. strategy/policy*⁶)
- Develop trustworthy systems

¹ SDLC – Software Development Lifecycle

² Annotation: SDLC is sometimes referred to the System Development Lifecycle (*e.g., by the U.S. Department of Justice – DOJ*)

³ Annotation: It might be useful to create suitable documentation about particular processes at the enterprise level

⁴ Annotation: A brief selection is covered in this lecture

⁵ cf. (Villarroel, Fernández-Medina, and Piattini, M., 2005)

⁶ cf. (ISO 9004)

GENERAL ROLES IN THE SDL(C)¹

SELECTED EXAMPLES

- **Builder.** Practice security engineering to build defensible systems – no holes.
- **Operations people.** Architect reasonable networks, defend/keep them up.
- **Administrators.** Understand distributed systems and activate least privilege.
- **Users.** The last bastion of defense understand software can be securely.
- **Executives.** Recognize that early security-investments affect the degree to which users² trust the products.

Software security is everyone's job in the SDL(C)

¹ cf. (McGraw, 2006, p. 38)

² Annotation: Users are either customers of the enterprise (i.e., external) or customers of business units (i.e., internal) (e.g. Controlling and IT)

SECURITY RESPONSIBILITIES IN THE SDL(C)

SELECTED EXAMPLES

- **Authorizing Official (AO).** Senior official or executive with responsibility for operating an information system at an acceptable level of risk.
- **Chief Information Officer (CIO).** IT-systems planning, acquisition.
- **Configuration Management (CM) Manager.** Managing effects of changes and configurations of systems/networks w.r.t. change management.
- **Contracting Officer.** Authority to deal with (*e.g. enter, terminate*) contracts.
- **Contracting Officer's Technical Representative.** A qualified employee manages technical aspects of contracts appointed by Contracting Officer.

¹ cf. (Kissel et al., 2008, pp. 9-10)

² Annotation: One can find other examples of project roles and responsibilities for instance in: pp. 205-206, Gregory, P. (2016). CISA Certified Information Systems Auditor Exam Guide.

SECURITY RESPONSIBILITIES IN THE SDL(C) | cont'd

SELECTED EXAMPLES

- **Information System Security Officer.** Responsible for ensuring the security of an information system throughout its life cycle.
- **Information Technology Investment Board (or equivalent).** Architect reasonable networks, defend/keep them up.
- **Legal Advisor/Contract Attorney.** Legal advice along acquisition process.
- **Privacy Officer.** Ensure the system meets privacy policies w.r.t. protection, dissemination (*information sharing and exchange*), and information disclosure.
- **Program Manager / Official (Information Owner).** Represents business and programmatic interests during the SDLC as an essential security role.

¹ cf. (Kissel et al., 2008, pp. 9-10)

² Annotation: One can find other examples of project roles and responsibilities for instance in: pp. 205-206, Gregory, P. (2016). CISA Certified Information Systems Auditor Exam Guide.

SECURITY RESPONSIBILITIES IN THE SDL(C) | cont'd

SELECTED EXAMPLES

- **QA/Test Director.** System test and evaluation. It serves across a variety of programs by assisting in the development and execution of test plans.
- **Senior Agency Information Security Officer (SAISO)².** Construct policies on the security integration in the SDLC and developing enterprise standards to identify, evaluate, and minimize information security risk.
- **Software Developer.** Programmatic “secure coding” to implement controls.
- **System Architect.** Overall designer and application integrator.
- **Other Participants.** The list grows with the complexity of the systems. It may include other representatives (*e.g. requirements engineers*).

¹ cf. (Kissel et al., 2008, pp. 9-10)

² Annotation: The SAISO is also known as CISO (*i.e., Chief Information Security Officer*)

BUILDING ASSURED SYSTEMS INTEGRATE SECURITY AND ASSURE IT

- **Engineering resilient systems.** Includes secure software engineering, requirements engineering, architecture, design, system operation continuity.
- **Containment.** Monitor/detect components behavior and recover from error.
- **Architecting secure systems.** Design artifacts, quality attributes, trade-offs, security properties positioning, the relation to the complete system.
- **Secure software engineering.** Includes e.g. secure coding, software engineering, hardware design improvement to reduce vulnerabilities. It covers technology lifecycle assurance² mechanisms, advanced engineering, standards, certification and best practices.

Software security is everyone's job in the SDL(C)

¹ cf. (Mead and Woody, 2016, p. 28)

² cf. (ISO/IEC/IEEE 12207, pp. 52-53)

CONSIDER SECURITY PRACTICES CONTINUOUSLY THE SYSTEM DEVELOPMENT

*“The system development life cycle (**SDLC**) is the overall process of developing, implementing, and retiring information systems.”¹*

*“The secure development lifecycle (**SDL**) comes from adding security [best practices²] to a SDLC to reduce the number of security bugs”³*

A variety of security-related frameworks and models exist

Think about security early and often during the software lifecycle

¹ cf. (Stallings, 2019, Chapter 8 ‘System Development’)

² cf. (McGraw, 2006, p. 240, p. 256)

³ cf. (Conklin and Shoemaker, 2013, p. 83)

SOFTWARE SECURITY FRAMEWORKS AND MODELS

Frameworks. *CERT-RMM-RTSE¹, DHS SwA², NIST-Cyber-Security-Framework, NIST SP 800-64, OWASP-CLASP³, SEI-BASF⁴*

Models. *BSIMM, CMMI PRM, MS-SDL, OWASP-SAMM⁵, SEI CMMI[®]*

- Activities can be both concurrent or iterative
- A set of best practices to identify management and technical measures
- Establish an organizational process throughout the SDLC
- Relevant focus areas are similar by comparing each model/framework

It does not guarantee that a software product is free of weaknesses

¹ CERT-RMM-RTSE – CERT Resilience Management Model Resilient Technical Solution Engineering Process Area

² DHS SwA – Department of Homeland Security Software Assurance (Measurement Work) .

³ OWASP-CLASP – Open Web Application Security Project Comprehensive, Lightweight Application Security Process.

⁴ SEI-BASF – Software Engineering Institute Building Assured Systems Framework

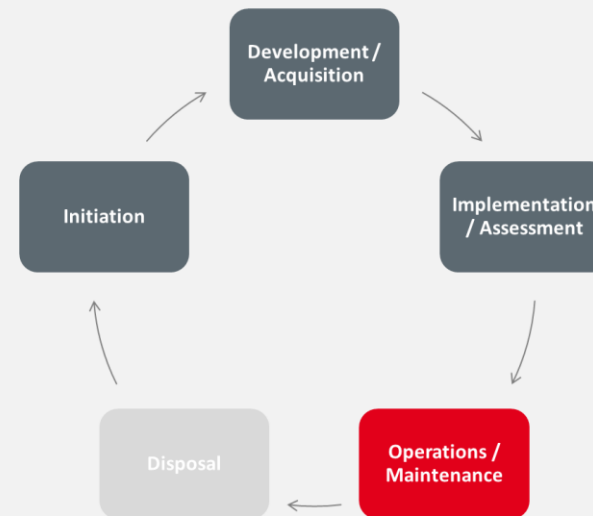
⁵ SAMM – Software Assurance Maturity Model .

NIST¹ SP² 800-64

SECURITY IN THE SYSTEM DEVELOPMENT LIFECYCLE

- **Applicable independently**
- Custom built system
- Made from scratch
- Purchased and customized
- COTS³ are covered
- **SDLC**
- **Security Considerations**

Exhibit | 5 SDLC-Phases



Source: NIST SP 800-64 | Software-Development-Lifecycle – Own figure

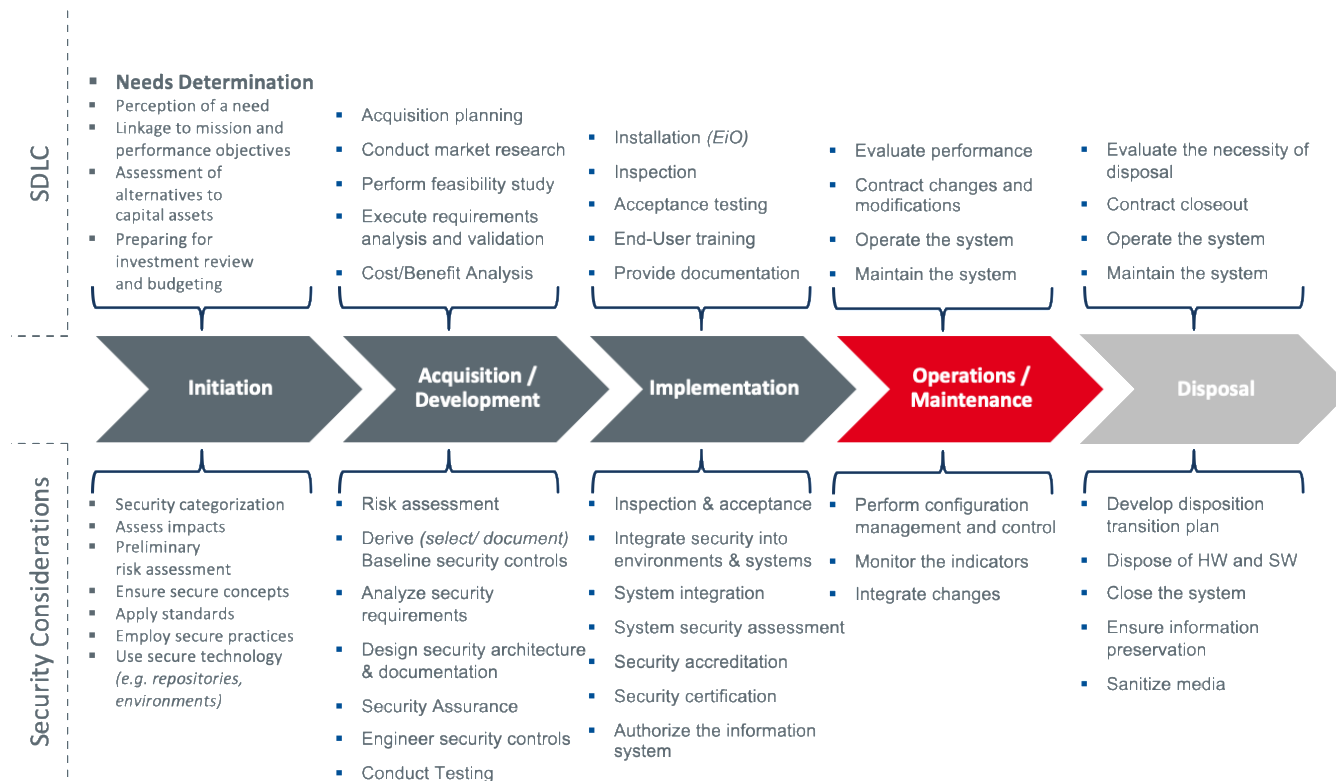
¹ NIST – National Institute of Standards and Technology

² SP – Special Publication

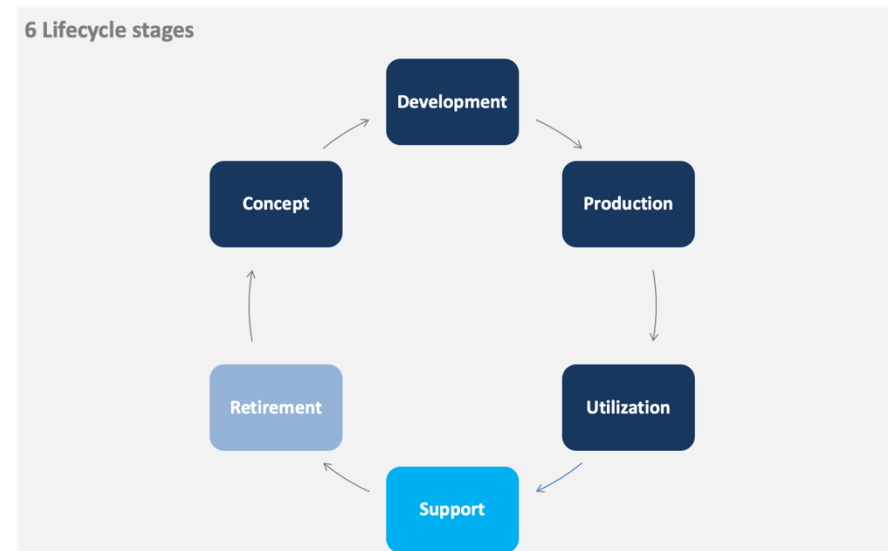
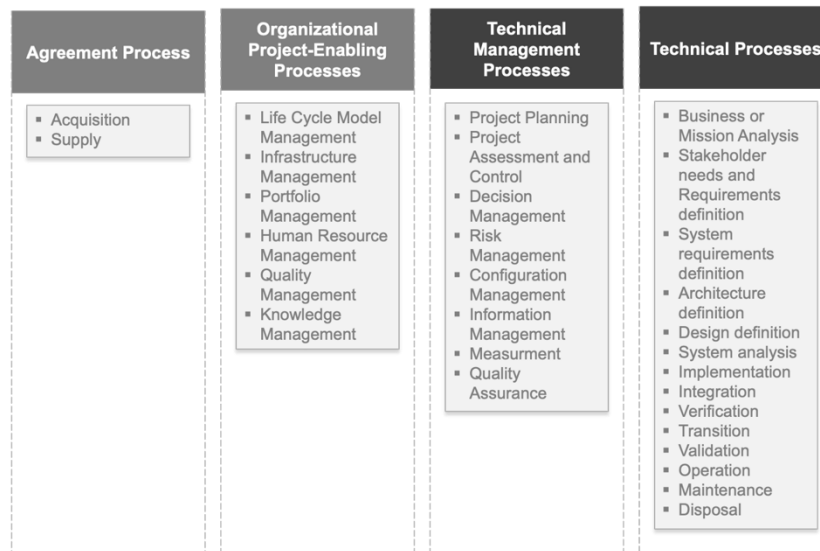
³ COTS – Commercial off-the-shelf (software)

NIST SP 800-64

SECURITY IN THE SYSTEM DEVELOPMENT LIFECYCLE



NIST SP 800-160 Vol. 1¹ SUCCESSOR OF NIST SP 800-64



Source: NIST SP 800-160 | Own figure based on (NIST SP 800-160, 2019) - Software-Development-Lifecycle stages

Original Source: ISO/IEC/IEEE 15288:2015

6 Lifecycle Stages applied to 4 System Life Cycle Processes

¹ cf. (NIST SP 800-160, 2019)

SGP¹ SDLC MODEL BY ISF²

2 SEPARATE PHASES AND 10 PROCESSES

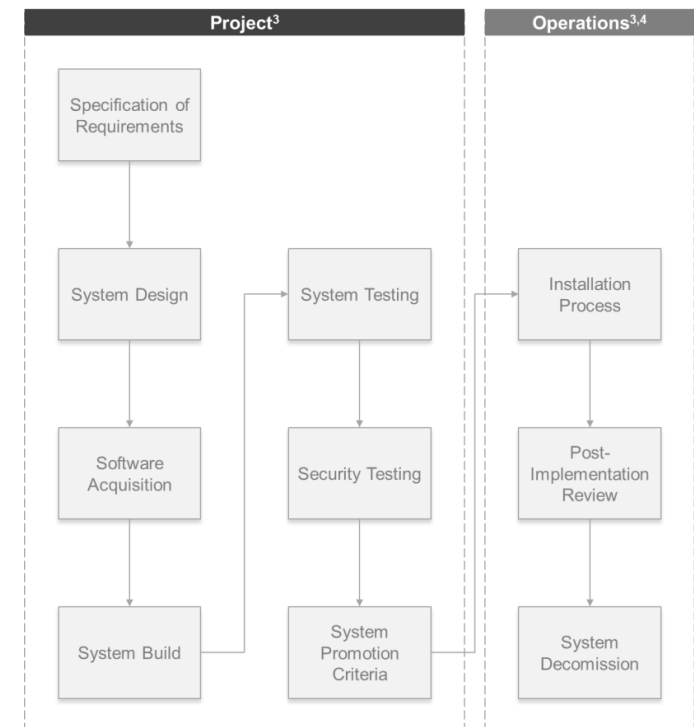
- Finer grained than NIST SP 800-64 model
- **Similar basic concepts**
- **Hardware** and/or **Software**
- **Supply chain considered**
- **Best practices** for development
- Consider information lifecycle

¹ SGP – Standard of Good Practice for Information Security

² ISF – Information Security Forum

³ Annotation: SGP model amended by G. Dißbauer based on project experiences with two distinct phases (*i.e. Project, Operations*)

⁴ Annotation: Operations typically can include also e.g.: maintenance, (*emergency*) change management, system disposal



Own Figure based on (Stallings, 2019)

OWASP CLASP¹

THE PREDECESSOR OF OWASP SAMM

*“OWASP CLASP is **[was]** the result of many development lifecycles were methodically decomposed in order to create a comprehensive set of security requirements.”²*

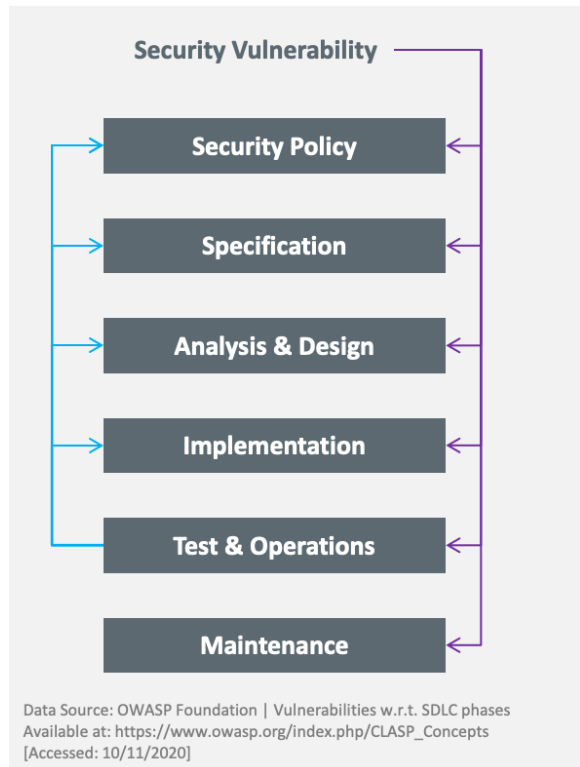
- It is now outdated and it was replaced/renewed by **OWASP SAMM**
- Integrate CLASP security-related activities and 7 key best practices early into your existing application development processes.
- Each CLASP activity is divided into discrete process components and linked to one or more specific project roles.

¹ CLASP – Comprehensive Light Weight Application Security Process

² Available at: https://www.owasp.org/index.php/CLASP_Concepts [Accessed: 02/01/2026]

OWASP CLASP | cont'd

7 CLASP BEST PRACTICES



7 CLASP Best Practices

- Establish awareness program
- Conduct application assessment
- Gather security requirements
- Set up secure development practices
- Build vulnerability remediation process
- Define and monitor metrics
- Publish operational security guidelines

Pre-operational entry points of vulnerability

Points in operational system attackers target

OWASP SAMM^{1,2}

SOFTWARE ASSURANCE | OPEN FRAMEWORK

Open framework to draw and implement a strategy to conduct software security engineering that is tailored to organizational-specific risks.

SAMM-Resources aid in³

- Evaluating an organization's existing software security practices
- Building a balanced software security assurance program in iterations
- Demonstrate concrete improvements to a security assurance program
- Defining and measuring security-related activities throughout an organization

OWASP-SAMM is based on OWASP CLASP and it is its successor

¹ cf. OWASP-SAMM – Open Web Application Security Project (OWASP) Software Assurance Model (SAMM)
Available at: https://www.owasp.org/index.php/OWASP_SAMM_Project [Accessed: 02/01/2026]

² More details available at: www.opensamm.org [Accessed: 02/01/2026]

³ cf. (OWASP-SAMM, no date, p. 3)

OWASP SAMM | cont'd

3 SAMM-PRINCIPLES

SAMM can be utilized by small, medium and large organizations using any style of (software) development

3 SAMM-Principles¹

- An organization's behavior changes slowly over time
A successful software security program should be specified in small iterations that deliver tangible assurance gains while incrementally working toward long-term goals.
- There is no single recipe that works for all organizations
A software security framework must be flexible and allow organizations to tailor their choices based on their risk tolerance and the way in which they build and use software.
- Guidance related to security activities must be prescriptive
All the steps in building and assessing an assurance program should be simple, well-defined, and measurable. This model also provides roadmap templates for common types of organizations.

OWASP SAMM | cont'd

THE STRUCTURE SUMMARIZED

A brief Summary of the SAMM-Structure¹

- 4 critical Business Functions at the highest level
Each business function is a category of activities related to software development
- 3 Security Practices for each Business Function
Each security practice is an area of security-related activities to build assurance for the associated business function as an improvement measure
- 3 Maturity Levels² for each Security Practice as Objectives
Each level is characterized by objectives defined by specific activities and success metrics³

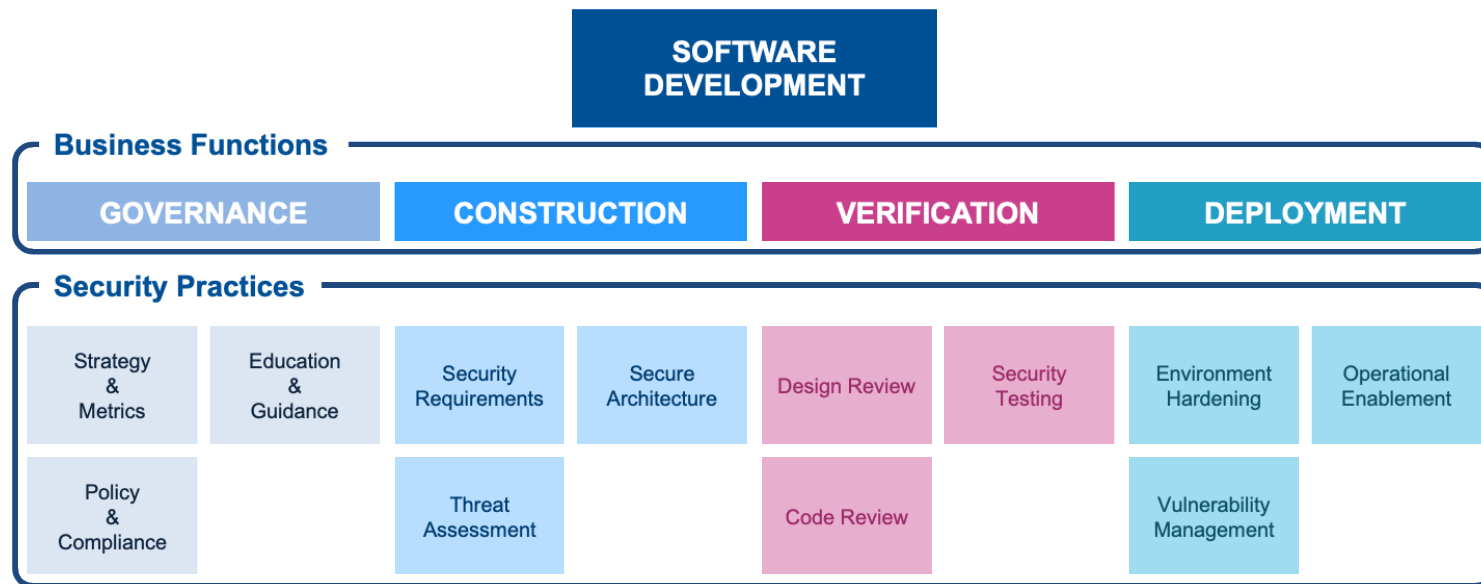
¹ cf. (OWASP-SAMM, no date, p. 3)

² Annotation: 3 particular maturity levels (i.e. 1, 2, 3) and one additional implicit starting point (i.e. 0).

³ Annotation: A metric is both (1) a quantity to measure a specific property and it is (2) a scale as well as a method used for measurements.

OWASP-SAMM | cont'd

The SAMM-Model (see figure below¹) can be utilized by small, medium and large organizations using any style of (software) development



¹ Own figure, Data Source: (OWASP-SAMM, no date, p. 3)

OWASP SAMM | cont'd

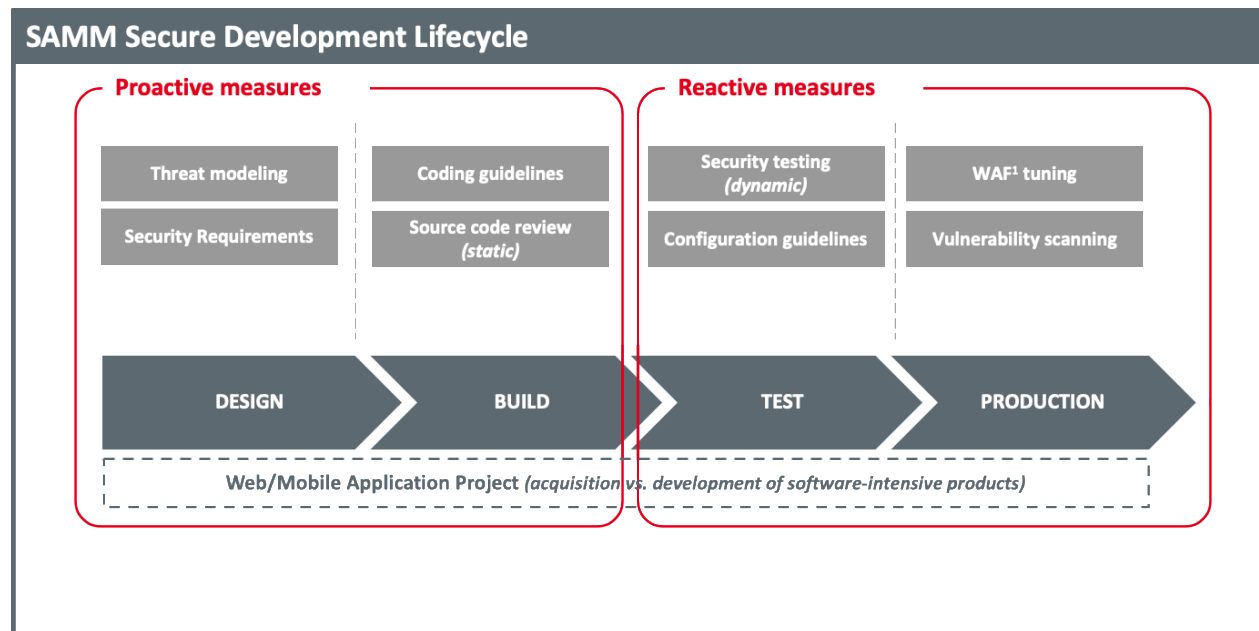
4 BUILDING BLOCKS

Governance	Construction	Verification	Deployment
Strategy & Metrics Overall strategic direction of the software assurance program & instrumentation of processes & activities to collect metrics about an organization's security posture	Threat Assessment Identify and characterize potential attacks on software to better understand the risks and facilitate risk management	Design Review Inspect artifacts created from the design process to ensure provision of adequate security mechanisms and adherence to expectations for security	Vulnerability Management Establish consistent process for managing internal and external vulnerability reports to limit exposure and gather data to enhance the security assurance program
Policy & Compliance Set up a security and compliance control framework and audit framework to achieve increased assurance in software under construction and in operation	Security Requirements Promote the inclusion of security-related requirements during the software development process to specify correct functionality from inception	Code Review Assess source code to aid vulnerability discovery and related mitigation activities as well as establish a baseline for secure coding expectations	Environment Hardening Implement controls for the operating environment in which software executes to bolster the security posture of applications that have been deployed
Education & Guidance Increase security knowledge among personnel in software development through training and guidance on security topics relevant to individual job functions	Secure Architecture Bolster the design process with activities to promote secure-by-default designs and control over technologies and frameworks upon which software is built	Security Testing Test software in its runtime environment in order to discover vulnerabilities and establish a minimum standard for software releases	Operational Enablement Identify and capture security-relevant information needed by an operator to properly configure, deploy, and run software

Own figure

OWASP SAMM | cont'd

SECURE DEVELOPMENT LIFECYCLE PROCESS

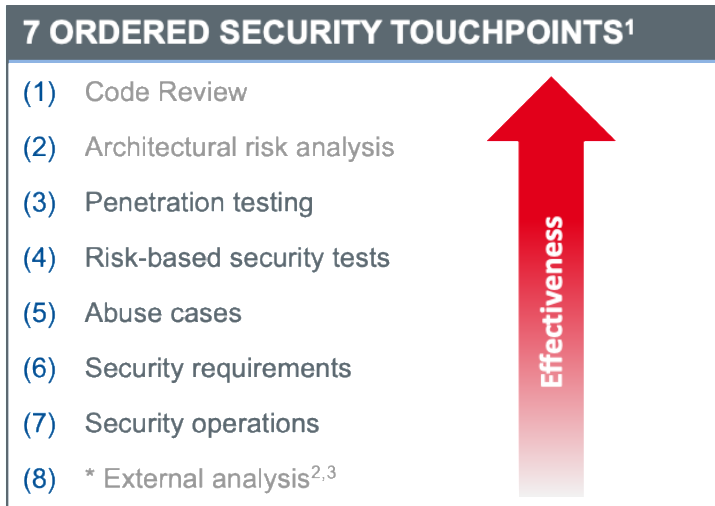
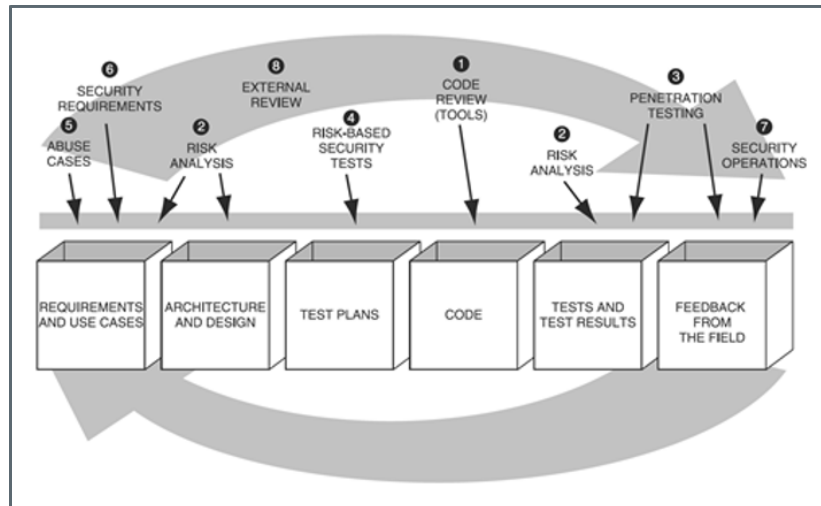


- Build in Software Assurance in the Software Development Lifecycle at early stages
- Apply both (1) proactive and (2) reactive measures to develop systems trustworthy

¹ WAF – Web Application Firewall

CIGITAL SECURITY TOUCHPOINTS¹

7+1 TOUCHPOINTS



- Apply security engineering best practices to artifacts and adopt the touchpoints
- Obtain priorities and decide the touchpoint ranking to remove bugs and flaws

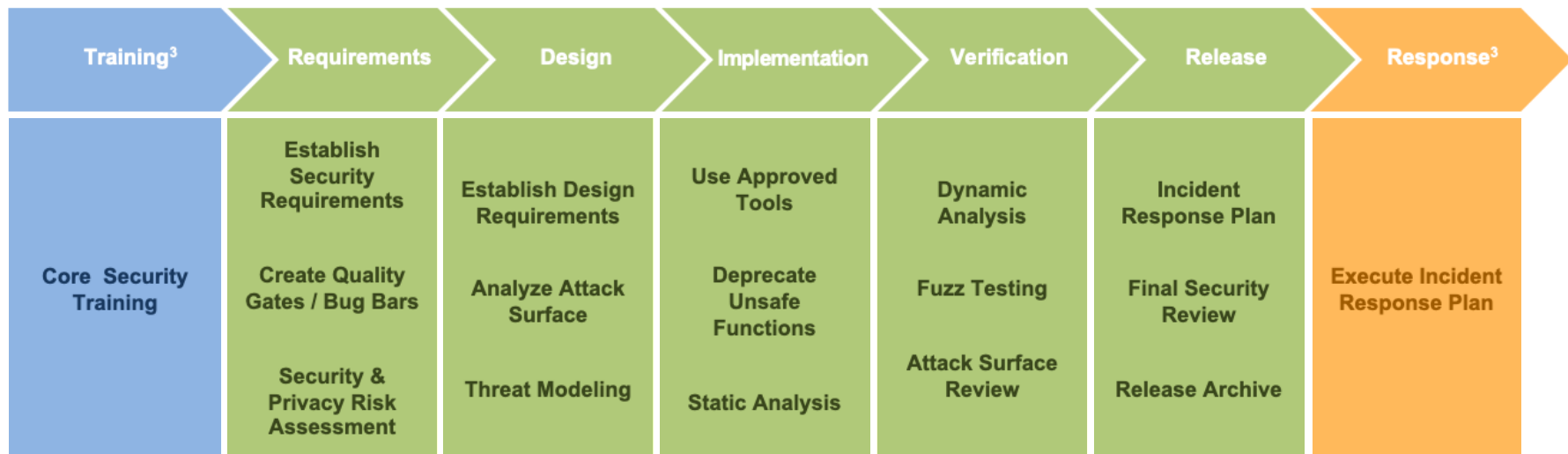
¹ cf. (McGraw, 2006, p. 84)

² Annotation: External analysis is important. However, it is not a particular touchpoint, but is mentioned in the big-picture of the touchpoints

³ cf. (McGraw, 2006, pp. 88-89)

MS-SDL^{1,2}

MICROSOFT SECURITY DEVELOPMENT LIFECYCLE



¹ cf. (Howard and Lipner, 2006)

² cf. Microsoft Security Development Lifecycle, Available at: <https://www.microsoft.com/en-us/sdl> [Accessed: 09/10/2021]

³ Annotation: Formally a pre-SDL-phase (*Training*) and a post-SDL-phase (*Response*)

4 GENERAL PRINCIPLES & 13 CONCEPTS | cont'd

MS-SDL SUMMARIZED

Secure by Design

Threat modeling and mitigation. Create threat

models and apply threat mitigation controls in all design and functional specifications

Elimination of vulnerabilities. Conduct both analysis and automatic reviews (e.g. *tools*) to remove all known vulnerabilities

Improvements in security. Minimize legacy technology (e.g. *code, protocols*) and provide secure solutions based on industrial standards

Secure by Default

Least privilege. Run all components with fewest feasible permissions

Defense in depth. Components do not rely on single-mitigation controls

Conservative default settings. Be aware about the attack surface for the product and reduce it in the default configuration.

Avoidance of risky default changes. Applications do not make any default changes to the operating system or security settings of the host (*Strengthen security sometimes allowed*).

Less commonly used services off by default. Deactivate unused¹ features by default.



¹ Annotation: Deactivate product-features that are used by fewer than approximately 80% of a systems users, by default

4 GENERAL PRINCIPLES & 13 CONCEPTS | cont'd

MS-SDL SUMMARIZED



Secure Deployment

Deployment guides. Prescriptive deployment guides outline how to deploy each feature of a program, securely.

Analysis and management tools. Security analysis and management tools enable administrators to determine and configure the security level for a software release

Patch deployment tools. Deployment tools aid in patch deployment

Communications

Security response. Development teams respond promptly to reports of security vulnerabilities and communicate information about security updates/patches.

Community engagement. Development teams proactively engage with users to answer questions in terms of security vulnerabilities, security updates/patches, or changes in the security landscape

MS-SDL | 7 PHASES

NO.1 TRAINING



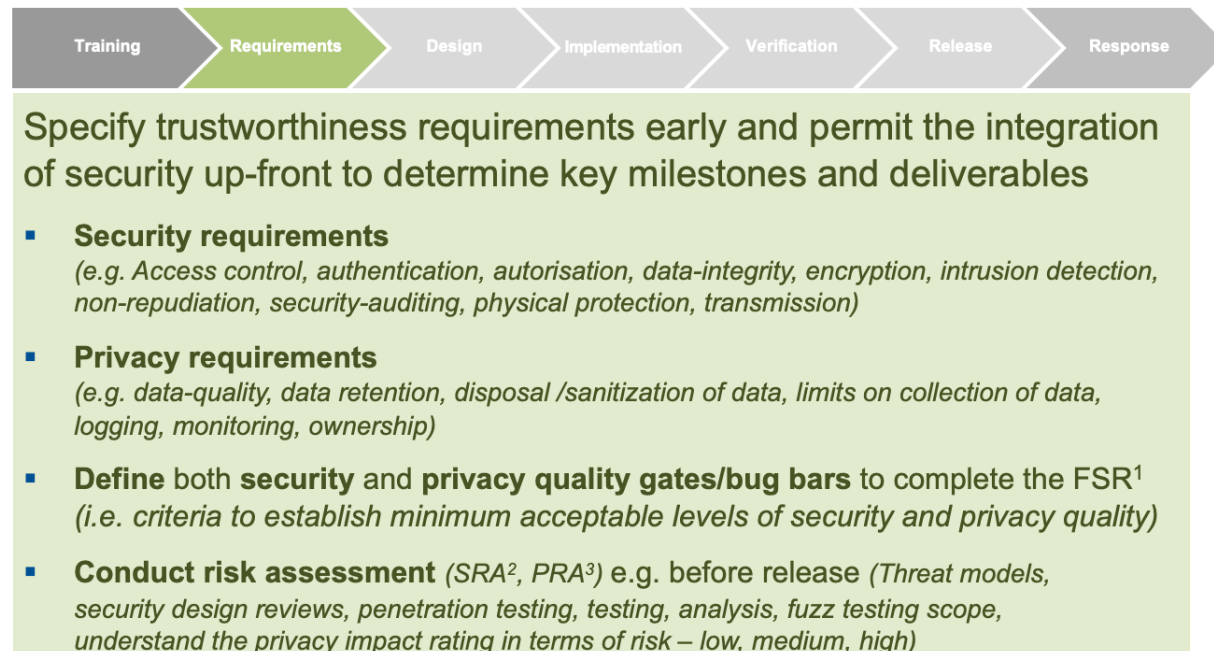
¹ XSS – Cross-site scripting

² SQL – Structured Query Language

³ Annotation: Where feasible means possible under reasonable conditions both in terms of know how and resources

MS-SDL | 7 PHASES

NO.2 REQUIREMENTS



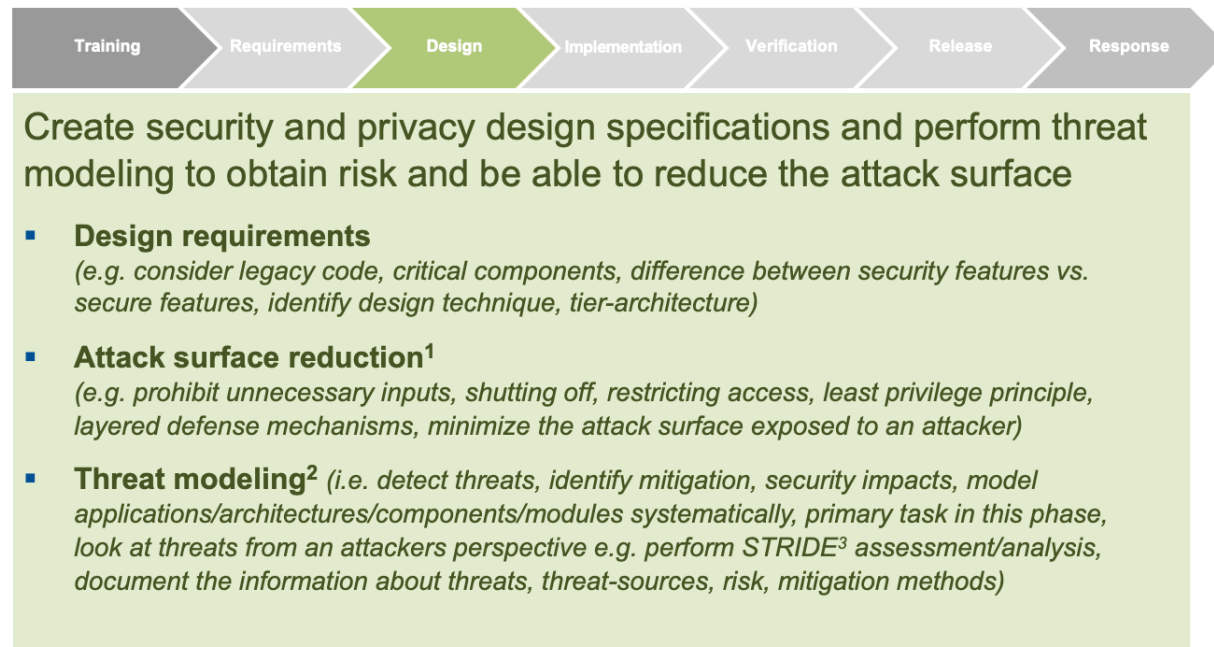
¹ FSR – Final Security Review

² SRA – Security Risk Assessment

³ PRA – Privacy Risk Assessment

MS-SDL | 7 PHASES

NO.3 DESIGN



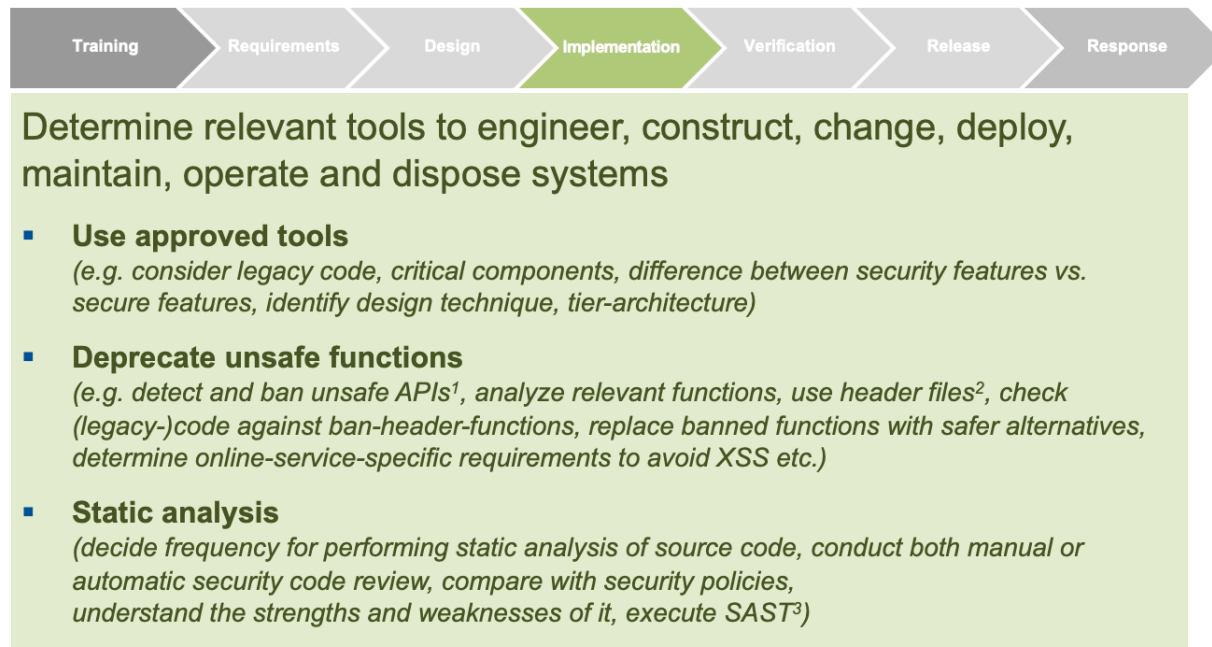
¹ Annotation: Every new feature of a software product increases both risk and complexity. Thus it is harder to defend.

² Annotation: It includes a lot of roles of the team *(i.e. project managers, software developers, test engineers)*

³ STRIDE – Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

MS-SDL | 7 PHASES

NO.4 IMPLEMENTATION



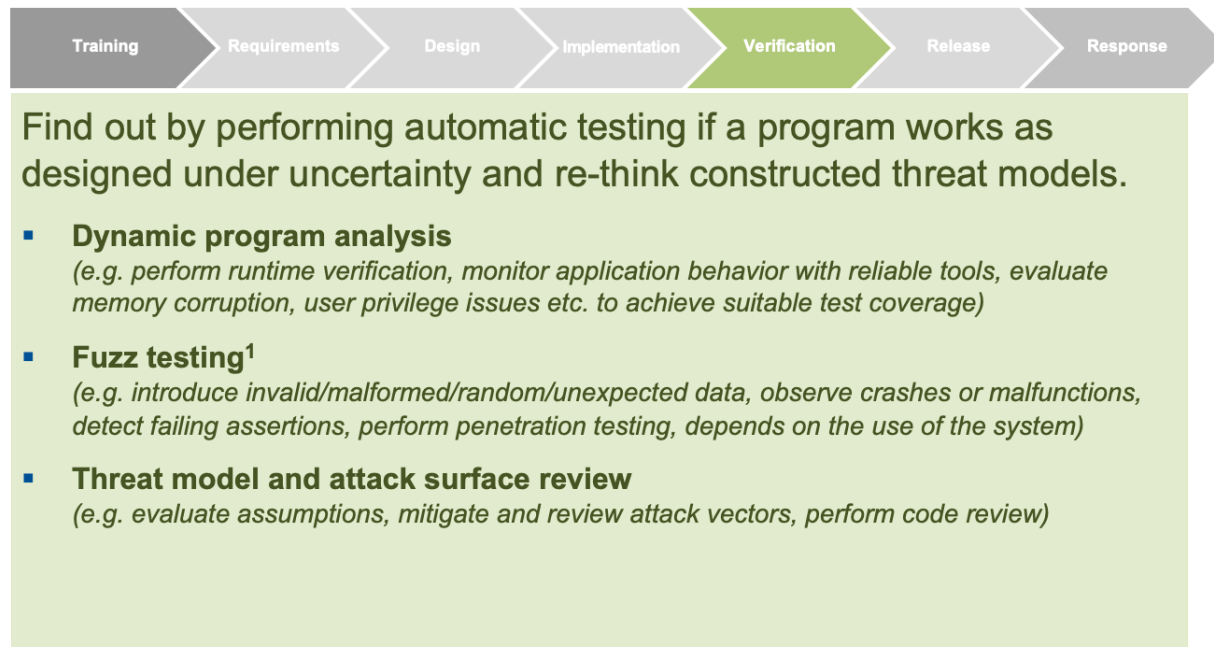
¹ API – Application programming interface

² Annotation: Examples are: banned.h, strsafe.h

³ SAST – Static Application Security Testing

MS-SDL | 7 PHASES

NO.5 VERIFICATION



¹ Annotation: An automatic software testing technique

MS-SDL | 7 PHASES

NO.6 RELEASE



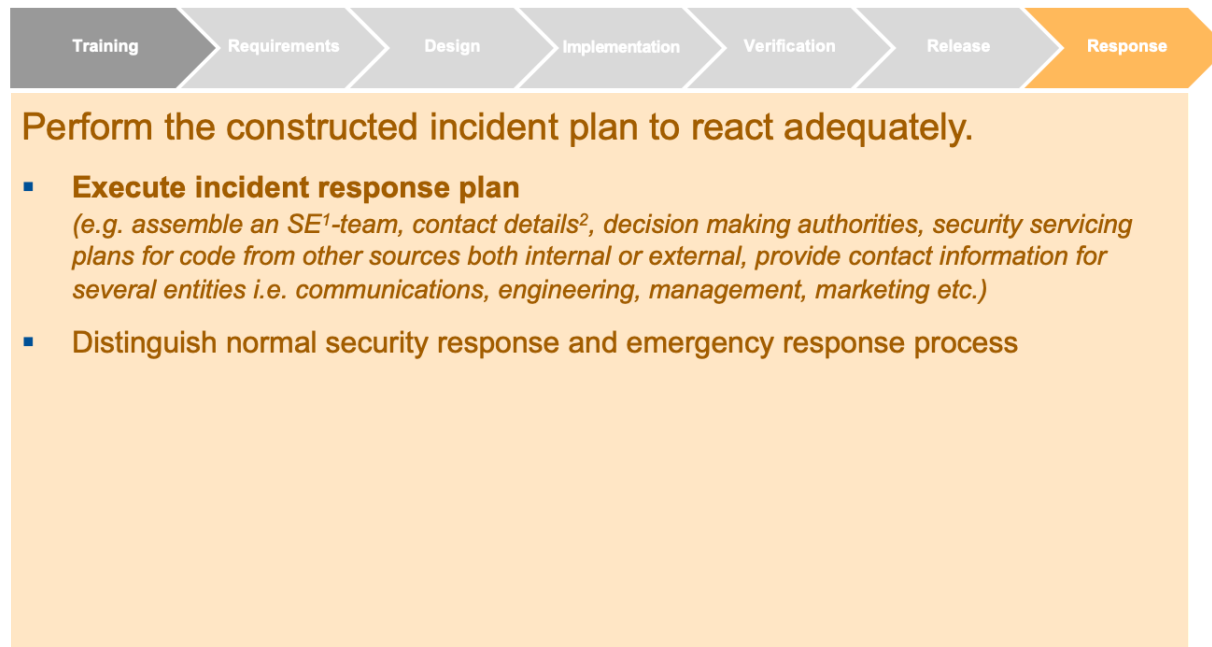
¹ SE – Sustained Engineering

² Annotation: SPOC system concerning both crisis or emergency handling particularly in terms of mission-critical systems EiO/changes etc.

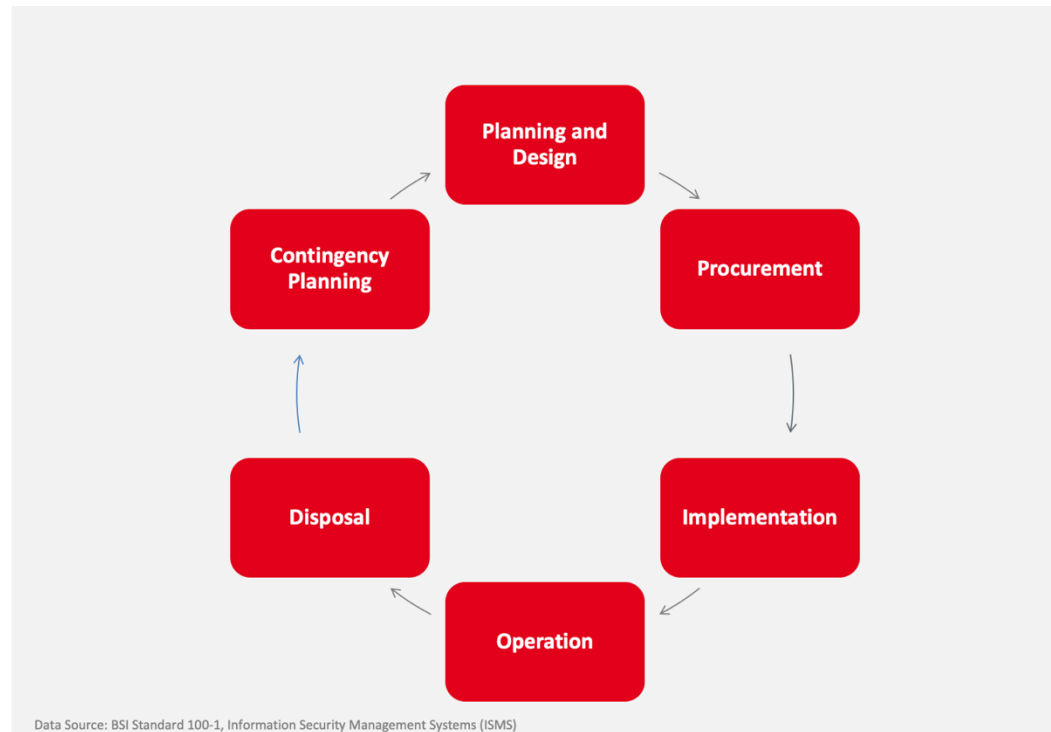
³ RTM – Release to Manufacturing and RTW – Release to Web

MS-SDL | 7 PHASES

NO.7 RESPONSE



BSI¹ 100-1 LIFECYCLE MODEL 6 PHASES



¹ cf. (BSI 100-1, p. 16)

COMMON CRITERIA LIFECYCLE MODEL

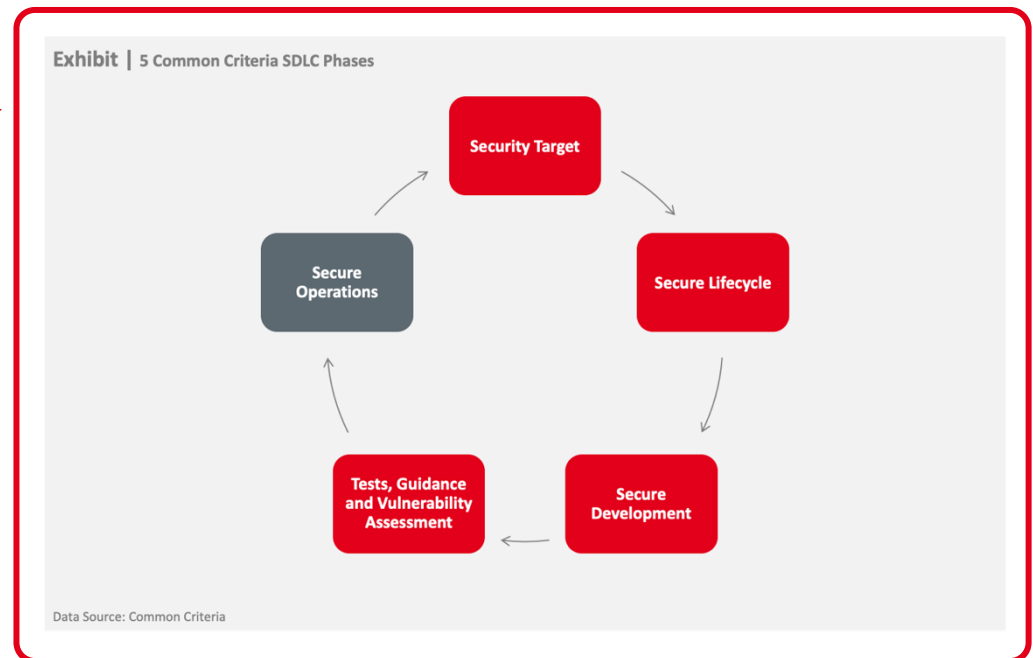
5 PHASES CONSOLIDATED

Common Criteria Lifecycle

- 5 phases

Correspond to Assurance Classes

- ASE Security Target Evaluation
- ADV Development
- AGD Guidance Documents
- ALC Lifecycle Support
- ATE Tests
- AVA Vulnerability Assessment
- ADV Development



LIFECYCLE OF MEDICAL INFORMATION SYSTEMS

PRACTICAL EXAMPLE | 5 PHASES | IEC 62304

5 Phases considered

Selected tasks to be performed in the lifecycle of Medical Information Systems

Software Development

- Development planning, Requirements engineering
- Software architecture, Software design, Implementation, Integration
- Software test, test planning, approval & release

Software Maintenance

- Operate the system, maintenance planning
- Status reporting, evaluate feedback
- Implement change requests, solve problems, verify solutions & release software

Software Risk Management

- Regulatory requirements, risk assessment matrix, risk acceptance, residual risks
- Conduct e.g., Preliminary Hazard Analysis (PHA), Fault Tree Analysis (FTA), FMEA (or FMVEA)
- Estimate probabilities (*damage, likelihood*), consolidate findings, create risk treatment plan

Configuration Management

- Identify configuration items, distinguish elements and versions
- perform version control and appoint versions
- Identify SOUP¹, perform change control (*assess/develop/test/approve/implement changes*)

Problem Solving

- Detect problems and report them
- Problem solving and verifying solutions
- Identify and analyze trends

¹ SOUP – Software of Unknown Provenance

LIFECYCLE OF MEDICAL PRODUCTS

PRACTICAL EXAMPLE | 4 additional PHASES | IEC 62304

4 additional phases

Product Risk Management

Selected tasks to be performed in the lifecycle of Medical Products

- Difference between medical devices/products and parts of medical devices/products
- Consider risks in the supply chain (e.g., manufacturer, SW-development, customer, hospital)
- Addressing both safety and security because human lives are strongly involved

Systemtest & Validation

- Test planning, select environments (e.g., 3-staging Development, Test, Production)
- Perform SW-Tests, HW-Tests (e.g., smoke test, integration, regression, software system)
- Test the integrated product (consider operational environment) and validate/verify changes & tests

Quality Management

- Surveillance and monitoring after going to market with the product
- Requirements of ISO 13485
- Quality management handbook, product realization, measurement, analysis and optimization

Document Management

- Create evidence and maintain it
- Document e.g., architecture, requirements and changes
- Make evidence accessible

Distinguishing (1) Medical Device or (2) Part of a Medical Device

¹ SOUP – Software of Unknown Provenance

Agenda

Introduction

IT Security in the Lifecycle of Software-intensive Products

Special Topics on Industrial Secure Lifecycle Practices

Practical Observations from Conducted Projects

Discussions & Summary

KEY PROBLEMS FROM PRACTICE

SELECTED EXAMPLES

- *„Organizations make or buy their technology [...] but rarely consider ways in which [...] it [...] functions within its intended deployment environment and the unintended consequences that are possible.“²*
- *„Every component of the software system and its interfaces must be operated and sustained with organizational risk in mind.“²*
- *„There is always uncertainty about a software systems behavior“³*
- *„At the start of a development cycle we have a limited basis for determining our confidence in the behavior of the delivered system“³*
- *“Virtually every connected system will be attacked sooner or later.”⁴*

¹ Annotation: Sometimes referred to Challenges

² cf. (Mead and Woody, 2016, p. 10)

³ cf. (Woody et al., 2014, p. 3)

⁴ cf. (Ebert, 2017, p. 209)

FEASIBLE SOLUTIONS FOR INDUSTRIAL PROBLEMS

SELECTED SUGGESTIONS

- **Risk-based Methodologies.** Informed decision-making
- **Security Processes.** Consider security with development processes
- **Security Requirements.** Blueprints for Software (*e.g. Protection-Profiles*)
- **Security Techniques.** Apply security solutions and security controls
- **Follow Best-Practices.** Use norms, standards, and best practices as baseline standards (*e.g. Common Criteria, DIN¹, ISO/IEC², IEEE³, MS-SDL, NIST, PCI-DSS⁴, SOG-IS-Crypto-Catalogue⁵*)
- **Continuous Improvement.** Efforts to achieve permanent optimization

¹ DIN – Deutsches Institut für Normung

² ISO/IEC – International Organization for Standardization International Electrotechnical Commission

³ IEEE – Institute of Electrical and Electronics Engineers

⁴ PCI-DSS – Payment Card Industry Data Security Standard

⁵ SOG-IS – Senior Officials Group Information Systems Security

LIFECYCLE OF A VEHICLE UNIT (VU) INDUSTRIAL EXAMPLE

Digital tachograph | A software-intensive system



Image Source: Siemens (2005).

The purpose of a VU is to (1) record, (2) store, (3) display, (4) print and (5) output data related to driver activities. It is a component of a Digital Tachograph System.

- Not every TOE has the same lifecycle process
- This product includes cryptographic operations (e.g. RSA)
- Assurance Class Life Cycle Support (ALC) included in PP

Anticipation w.r.t. Common Criteria Protection Profiles

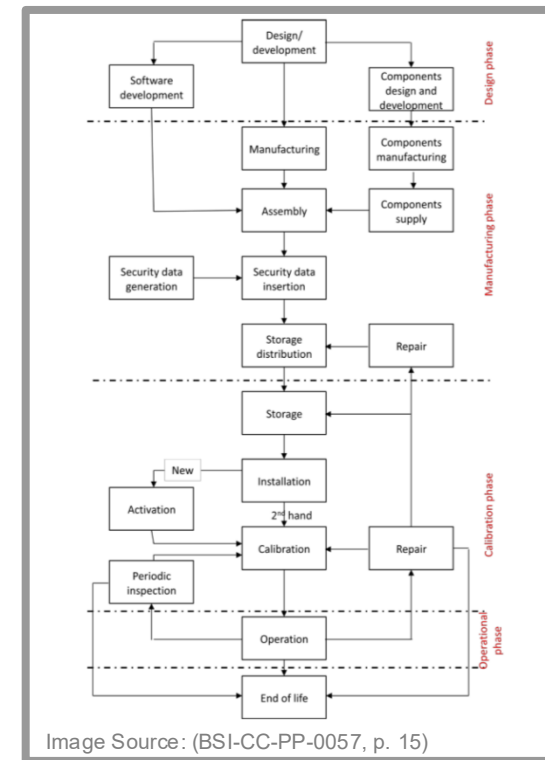


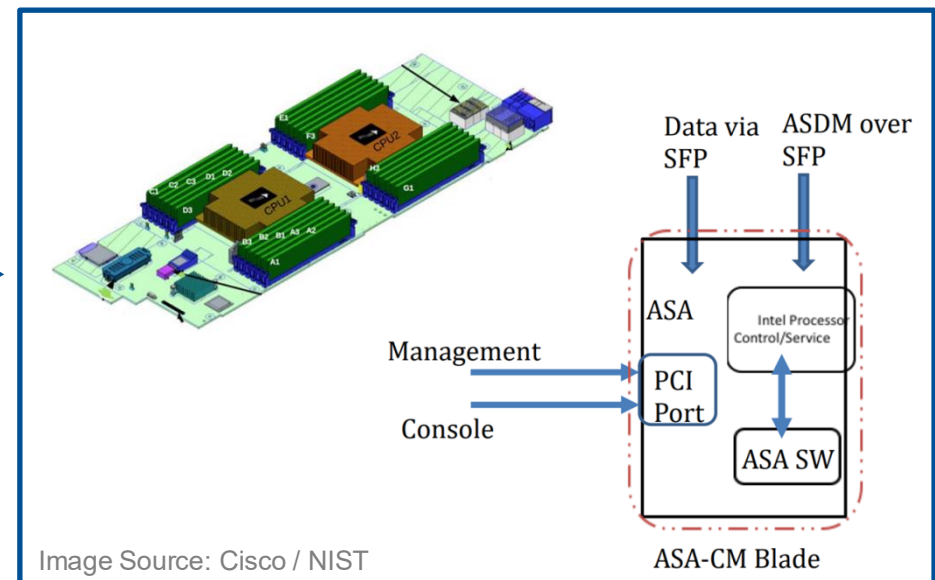
Image Source: (BSI-CC-PP-0057, p. 15)

CRYPTOGRAPHIC MODULE

EXAMPLE SOFTWARE-INTENSIVE PRODUCT

Cryptographic module¹:

The set of hardware, software, and/or firmware that implements approved security functions (*including cryptographic algorithms and key generation*) and is contained within the **cryptographic boundary**.



THE LIFECYCLE DURATION VARY SMARTPHONE VS. SMARTCARD

Selected comparision

- 1-3 years (*Smartphone*)
- 3-10 years (*Smartcard*)



The general duration time affects the particular lifecycle

SELECTED CRYPTOGRAPHIC TECHNIQUES

OVERVIEW | FIPS 140-2

- Symmetric key algorithms (*AES¹, EES², TDEA³*)
- Asymmetric key algorithms (*DSA⁴, ECDSA⁵, RSA⁶*)
- Message authentication methods (*CBC-MAC⁷, CCMP⁸, GMAC⁹, HMAC¹⁰*)

¹ AES – Advanced Encryption Standard

² EES – Escrowed Encryption Standard

³ TDEA – Triple Data Encryption Algorithm – Alternatively known as Triple DES (*3DES*) or Triple DEA

⁴ DSA – Digital Signature Algorithm

⁵ ECDSA – Elliptic Curve Digital Signature Algorithm

⁶ RSA – Rivest Shamir Adleman

⁷ CBC-MAC – Cipher Block Chaining Message Authentication Code

⁸ CCMP – Counter Mode Cipher Block Chaining Message Authentication Code Protocol or Counter Mode CBC-MAC Protocol

⁹ GMAC – Galois Message Authentication Mode

¹⁰ HMAC – Keyed Hashing for Message Authentication

SELECTED CRYPTOGRAPHIC TECHNIQUES | cont'd

OVERVIEW | FIPS 140-2

- Hash functions (*SHA-224, SHA-256, SHA-384, SHA-512¹*)
- Random number generators (*RNG² for DSA/ECDSA/RSA*)
- Deterministic random bit generators (*DRBG³ w.r.t. NIST SP 800-90A Revision 1*)
- Key management (*KAS ECC⁴ w.r.t. NIST SP 800-56A Revision 3*)

¹ SHA – Secure Hash Algorithm | Annotation: The mentioned hash functions represent agreed hash functions by SOG-IS to date.

² RNG – Random Number Generator

³ DRBG – Deterministic Random Bit Generator

⁴ KAS ECC – Key Agreement Schemes Elliptic Curve Cryptography

Agenda

Introduction

IT Security in the Lifecycle of Software-intensive Products

Special Topics on Industrial Secure Lifecycle Practices

Practical Observations from Conducted Projects

Discussions & Summary

ATTACK SURFACES OF SOFTWARE-INTENSIVE SYSTEMS

SELECTED PRACTICAL EXAMPLES

Selected elements that are practical instances of attack surfaces

- Cookies
- Enabled accounts (*e.g. guest, root*)
- External system calls (*e.g. RPC¹*)
- Network protocols (*e.g. UDP² vs. TCP³*) or open ports and/or sockets
- Shared resources (*e.g. databases, files*)
- Systems (*running, legacy*)
- Unused features or questionable input-types

¹ RPC – Remote Procedure Call

² UDP – User Datagram Protocol

³ TCP – Transmission Control Protocol

CRITICAL SUCCESS FACTORS IN THE SECURE LIFECYCLE

SELECTED EXPERIENCES CONSOLIDATED

- Appropriateness of processes based on international standards, systems and methods remains priority
- Business know how and technical expertise
- Consider costs and risks to make informed decisions
- Create suitable documentation that provides value
- Integrate security components to address risk management
- Live an organizational risk culture and create a risk policy
- Negotiate Management support and derive a strategy
- Obtain awareness that software security is everyones job in the SDLC
- Target to achieve that functional code behaves securely under qualitative requirements

PRACTICAL OBSERVATIONS

DERIVED FROM CONDUCTED PROJECTS

- The **lifecycle** of products **vary**
- Software development **models** and **frameworks vary**
- Both **data** and **systems/products have a lifecycle**
- Taking security into account both early and continuously can be improved in practice
- **Security** is often **conducted** as a **single project**
- **100% security does not exist**
- **Critical applications** are required to **fulfill** a lot of **security requirements**

Agenda

Introduction

IT Security in the Lifecycle of Software-intensive Products

Special Topics on Industrial Secure Lifecycle Practices

Practical Observations from Conducted Projects

Discussions & Summary

SUMMARY

COVERED TOPICS

- **Explained IT-security** concerns in the **lifecycle** of software-intensive products
- **Clarified the foundations of secure software lifecycles**
- **Explored secure lifecycle models**
- **Discussed practical relevance of IT security** in the software lifecycle
- **Identified particular techniques and security controls**
- **Discussed experiences** from conducted projects

